CS5330: Pattern Recognition and Computer Vision

Project 3 Report: Real-Time 2-D Object Recognition

Ahilesh Vadivel - 002055401

Description:

This project implements a real-time 2D object recognition system using a downward-facing camera feed streamed from a mobile phone via IP Webcam. The system identifies objects placed on a flat white surface in a translation, scale, and rotation invariant manner.

The pipeline consists of several sequential stages. First, a dynamic ISODATA thresholding algorithm separates objects from the background using HSV color space. The binary image is then cleaned using morphological opening and closing operations. Connected components analysis segments the image into distinct regions, filtering out noise and border-touching regions. For each valid region, geometric features are computed, including the axis of least central moment, oriented bounding box, percent filled, height/width ratio, eccentricity, and Hu moments, all of which remain stable under translation, rotation, and scaling.

Two classification systems were implemented and compared. The first uses scaled Euclidean nearest-neighbor matching on the five hand-crafted geometric features. The second is a one-shot embedding classifier using a pre-trained ResNet18 network, extracting 512-dimensional embeddings from the global average pooling layer. Performance was evaluated using confusion matrices for both systems. The hand-crafted system achieved 100% accuracy on the test set, outperforming the embedding system which achieved 62.5%, primarily due to visual similarity between certain objects.

Task 1: Threshold the input video

[Original Image] [Blurred Image] [Threshold Image]

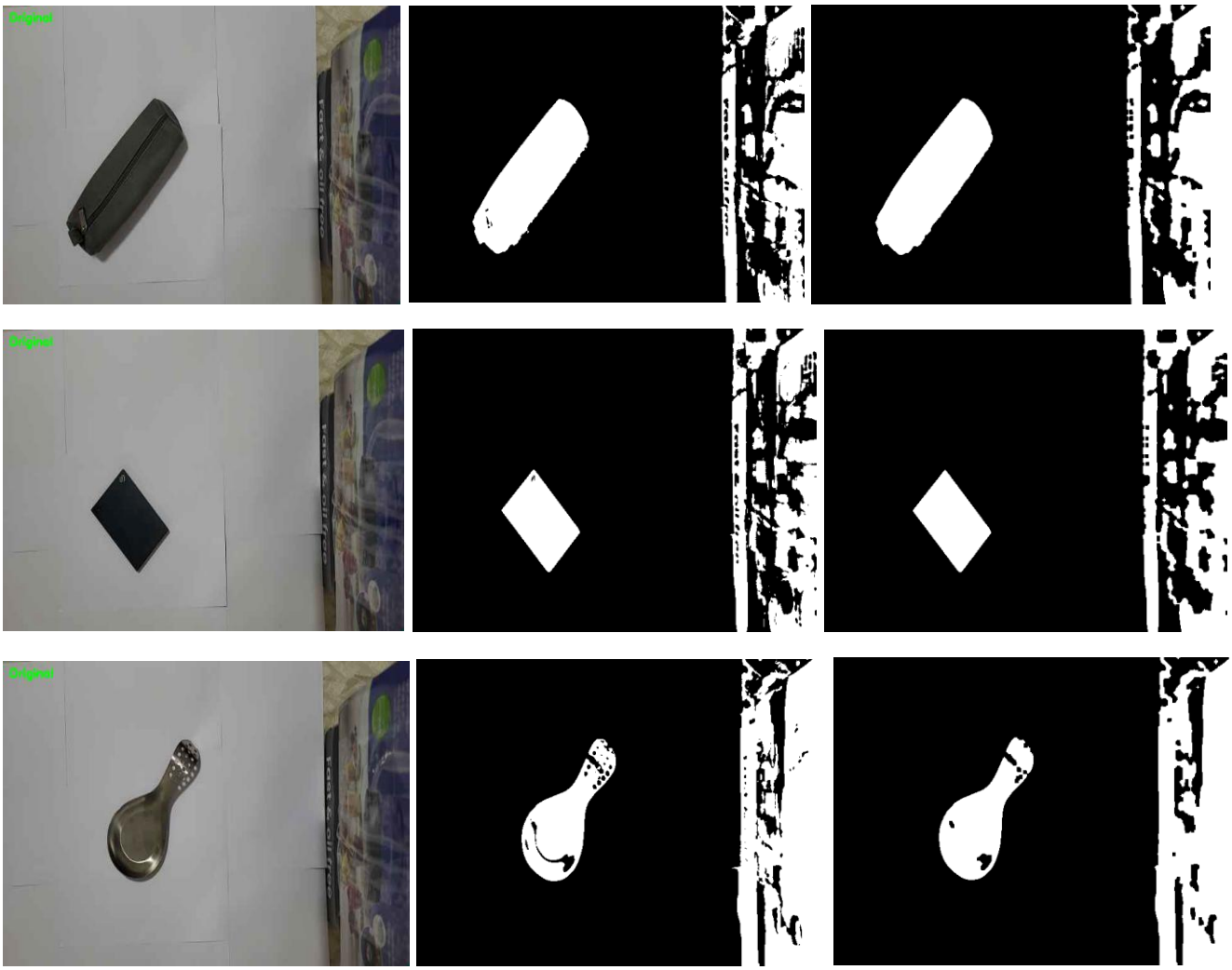
The figure shows thresholding results for three objects (pouch, hard disk, and vessel) displayed across three columns: the original frame, the Gaussian blurred frame, and the resulting binary image.

Gaussian blur is applied first to reduce noise and create more uniform regions. The ISODATA algorithm then dynamically finds the threshold by locating the midpoint between the two dominant brightness clusters in the image. A custom HSV threshold marks each pixel as foreground if its brightness falls below this threshold or its saturation exceeds a set limit.

In all three cases the object is correctly separated from the background as a solid white region against black. The pouch and hard disk threshold cleanly, while the vessel shows minor noise around its metallic handle due to uneven light reflection. Background clutter visible on the right edge is handled in the subsequent morphological cleanup step.

Task 2: Clean up the binary image

[Original Image] [Raw Threshold Image] [Cleaned Threshold Image]

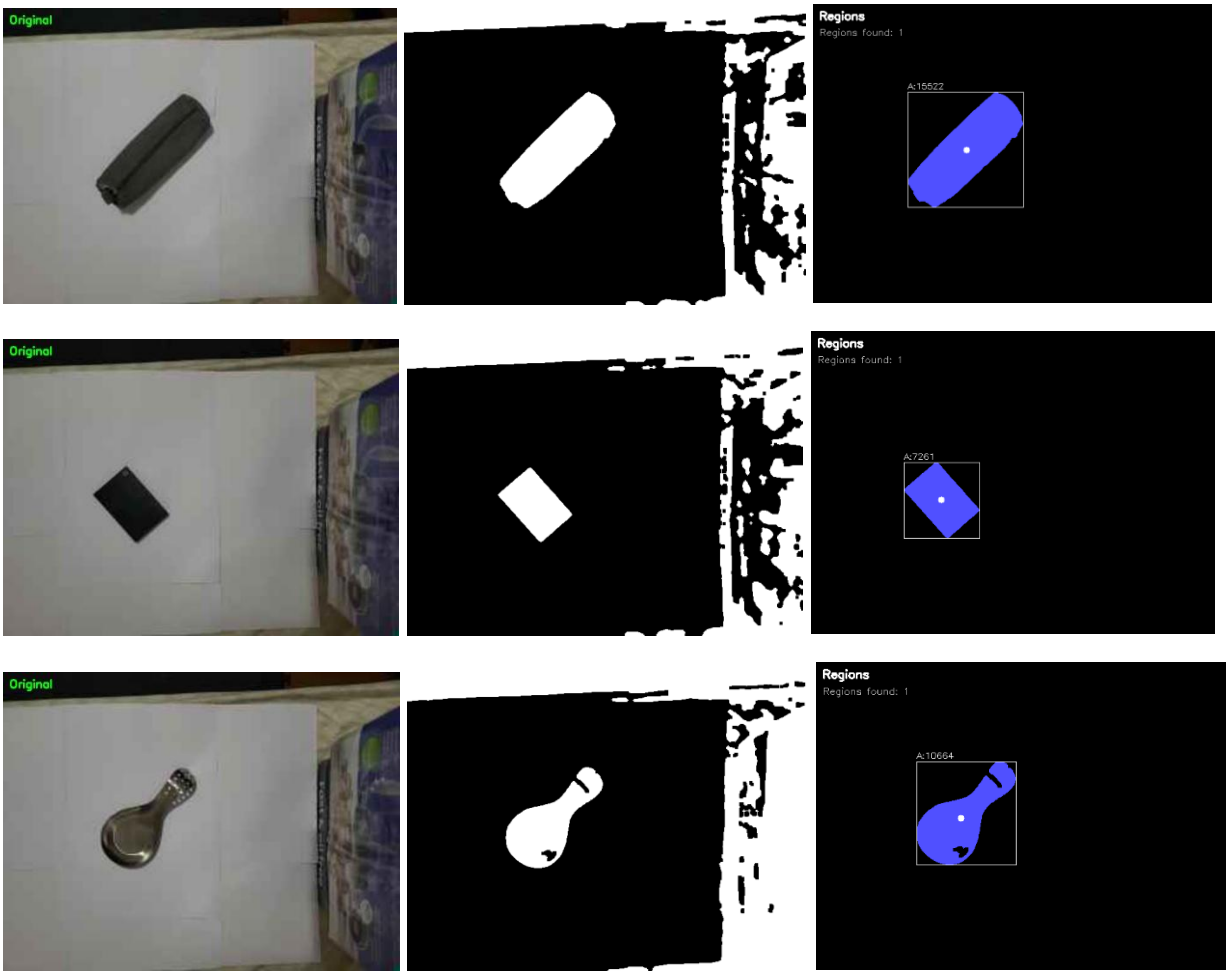


Problem observed: After thresholding, the binary images showed two issues, small white speckles scattered across the background (noise from lighting variation), and occasional small holes inside the object region caused by slightly reflective areas on the object surface.

Strategy chosen: Opening followed by Closing with 5x5 structuring elements. Opening (erode with a 4-connected cross kernel, then dilated with an 8-connected rectangular kernel) was applied first to remove the background noise without significantly affecting the object. Closing (dilate then erode with alternating kernels) was then applied to fill the interior holes. The alternating 4/8-connected kernel approach follows the textbook recommendation for better shape preservation.

Task 3: Segment the image into regions

[Original Image] [Cleaned Threshold Image] [Threshold Region Image]



[Multiple Objects Example (Different threshold regions represented with different colors)]

The figure shows segmentation results for three objects across three columns: the original frame, the cleaned binary image, and the colored region map.

Connected components analysis labels each group of connected foreground pixels with a unique region ID. Regions that are too small or touch the image border are filtered out, and the remaining valid regions are assigned distinct colors from a fixed palette. A white bounding box and centroid dot are overlaid on each detected region, along with its pixel area.

In all three single object cases exactly one valid region is detected and colored blue, correctly isolating the pouch, hard disk, and vessel from the background. The area value displayed above each bounding box confirms the region was successfully identified.

The bottom row demonstrates the extension to multiple objects, where three objects placed simultaneously in frame are each assigned a different color (red, green, and blue) and detected as separate regions. This confirms the system can segment and track multiple objects independently in a single frame.

Task 4: Compute features for each major region

Moments Used,

Features were computed using OpenCV's cv::moments() function on an isolated binary mask of each region. The function returns three families of moments:

- Spatial moments (m00, m10, m01...): computed relative to the image origin. Not translation invariant. Used only to locate the centroid.
- Central moments (mu20, mu11, mu02...): computed relative to the region centroid, making them translation invariant. Used directly to compute the central axis angle and eccentricity.
- Normalized central moments (nu20, nu11, nu02...): central moments divided by the power of the area, giving scale invariance in addition to translation invariance. Used to compute Hu moments H1 and H2.

Axis of Least Central Moment,

- The central axis angle alpha is the orientation of the axis along which the pixel distribution is most tightly concentrated. It is computed as:

  alpha = 0.5 * atan2(2 * mu11, mu20 – mu02)

- The major axis unit vector is vmajor = (cos(alpha), sin(alpha)) and the minor axis is vminor = (-sin(alpha), cos(alpha)). These axes form a coordinate system that rotates with the object. The axis is displayed as a yellow line through the centroid in the output.

Feature Vector,

Five features are computed per region. All are translation, scale, and rotation invariant.
1. Height/Width Ratio: Ratio of OBB height (major axis extent) to OBB width (minor axis extent). Describes elongation. A circle gives 1.0, a long rod gives a much larger value. Scale invariant because it is a ratio. Rotation invariant because the OBB aligns to the object.

2. Percent Filled: Ratio of the region's pixel area to the total OBB area. Describes compactness. A solid disc scores 0.78, a hollow or irregular shape scores lower. Invariant because both the pixel area and OBB area scale proportionally together.

3. Eccentricity: Derived from the eigenvalues ($\lambda 0$, $\lambda 1$) of the second-order central moment matrix:

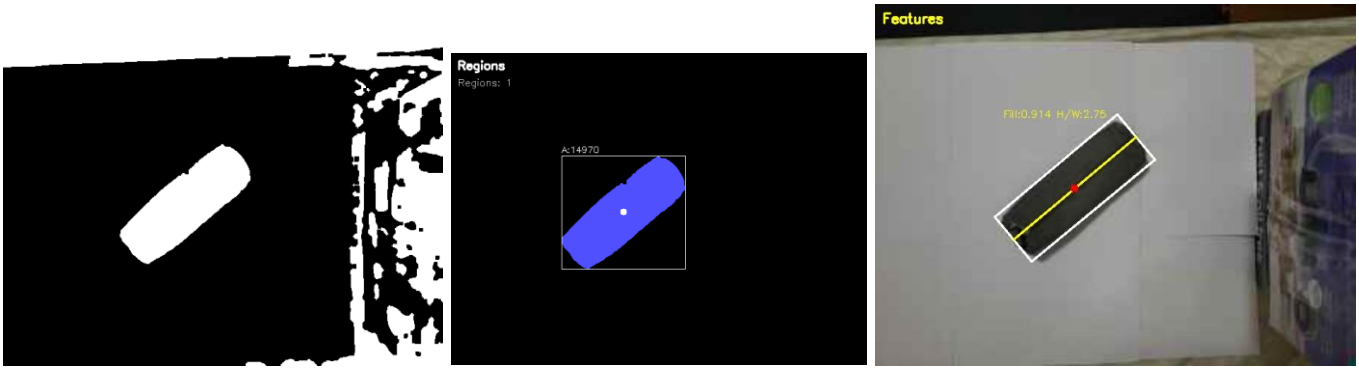
eccentricity = sqrt(1 - $\lambda 1$ / $\lambda 0$)

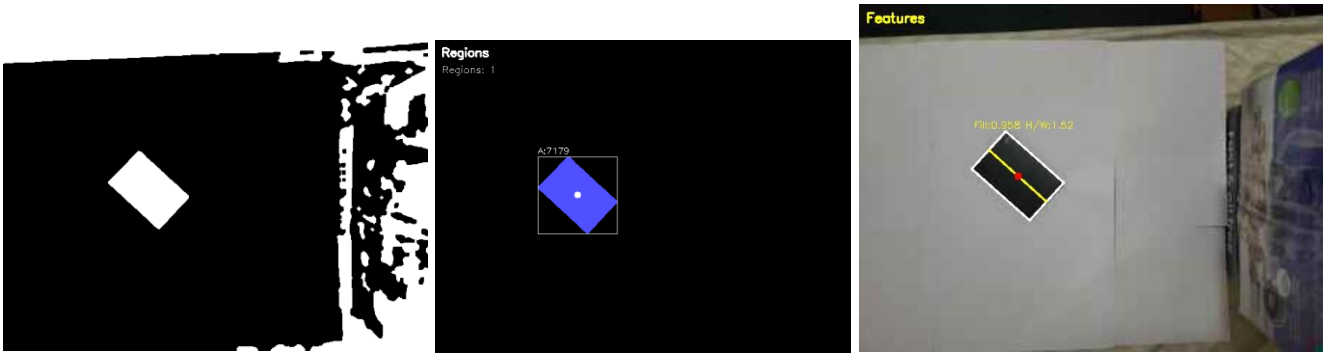
Ranges from 0 (circle) to 1 (line). Invariant because eigenvalue ratios are unaffected by translation, scale, or rotation.

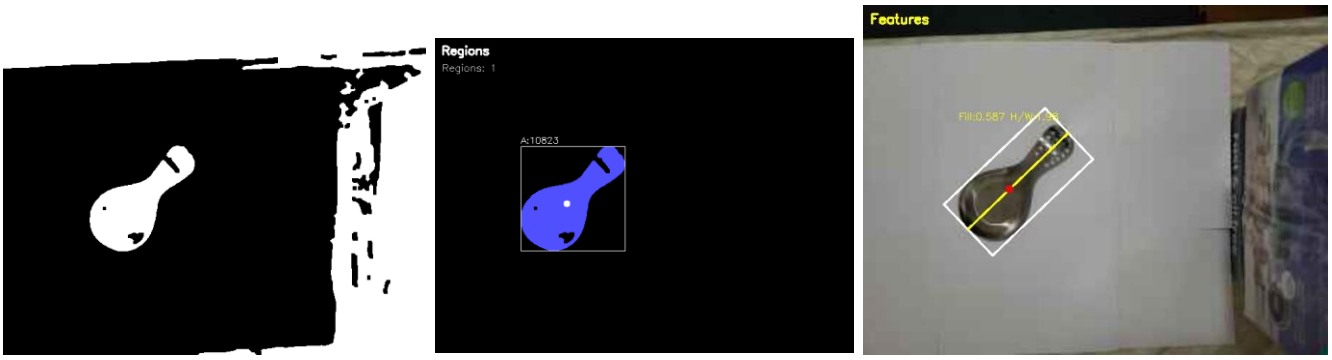
4. Hu Moment H1: H1 = eta20 + eta02
Captures the overall spatial spread of the region relative to its size. Compact shapes have a small H1, spread-out shapes have a larger value.

5. Hu Moment H2: H2 = (eta20 - eta02)^2 + 4 * eta11^2
Captures directional asymmetry. Equals 0 for rotationally symmetric shapes and grows as the shape becomes more elongated in one direction.

[Cleaned Threshold Image] [Threshold Region Image] [Axis of Least Central Moment & OBB]







```
--- Feature Vectors (save 0) ---
Region 1:
  Alpha (rad):    -0.69257
  H/W Ratio:      2.75926
  Percent Filled: 0.914483
  Eccentricity:   0.934606
  Hu Moment H1:   0.257477
  Hu Moment H2:   0.0398584
Saved: orig_0.png, cleaned_0.png, regions_0.png, features_0.png

--- Feature Vectors (save 1) ---
Region 1:
  Alpha (rad):    0.730283
  H/W Ratio:      1.5211
  Percent Filled: 0.9584
  Eccentricity:   0.756891
  Hu Moment H1:   0.182054
  Hu Moment H2:   0.00534094
Saved: orig_1.png, cleaned_1.png, regions_1.png, features_1.png

--- Feature Vectors (save 2) ---
Region 1:
  Alpha (rad):    -0.75891
  H/W Ratio:      1.98029
  Percent Filled: 0.587723
  Eccentricity:   0.914364
  Hu Moment H1:   0.281075
  Hu Moment H2:   0.0407625
Saved: orig_2.png, cleaned_2.png, regions_2.png, features_2.png
```

The figures on the right show the axis of least central moment and oriented bounding box overlaid on three objects: the pouch, hard disk, and vessel.

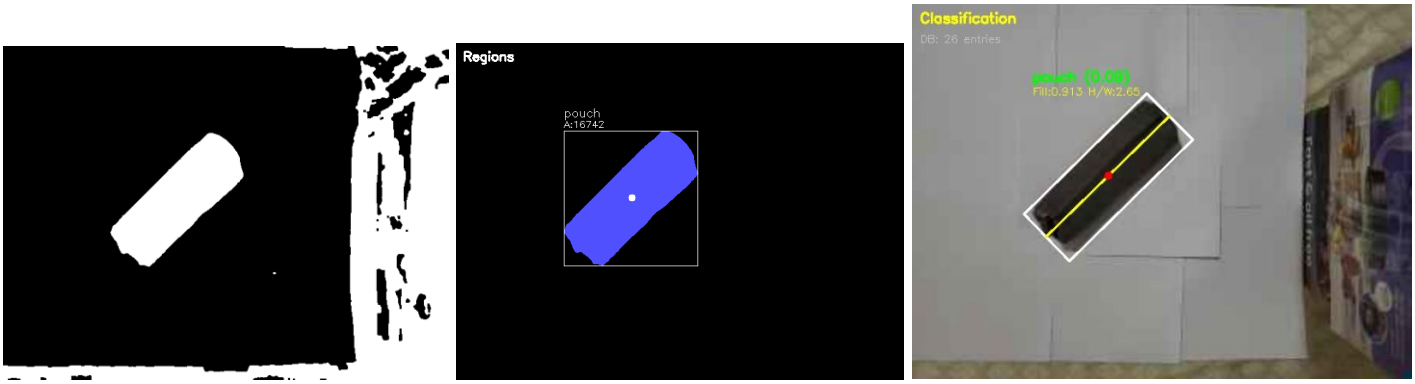
In each image the yellow line represents the primary axis, which passes through the centroid (shown as a red dot) and aligns with the direction of least central moment. The white rotated rectangle is the oriented bounding box, which is the tightest possible rectangle aligned to the object's own axis rather than the image frame. Both the axis and bounding box visibly rotate with the object regardless of its orientation, confirming rotation invariance. The percent filled and H/W ratio values are overlaid in yellow text on each region for real time inspection.

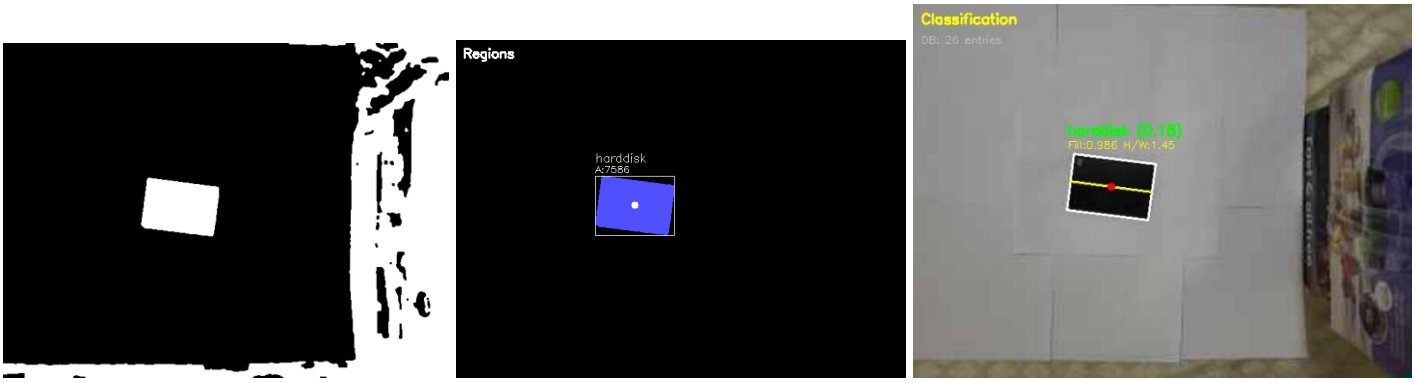
The pouch shows a clearly elongated bounding box with a high H/W ratio reflecting its long rectangular shape. The hard disk shows a more compact bounding box with a lower H/W ratio. The vessel shows an irregular shape with lower percent filled due to its non-uniform silhouette.

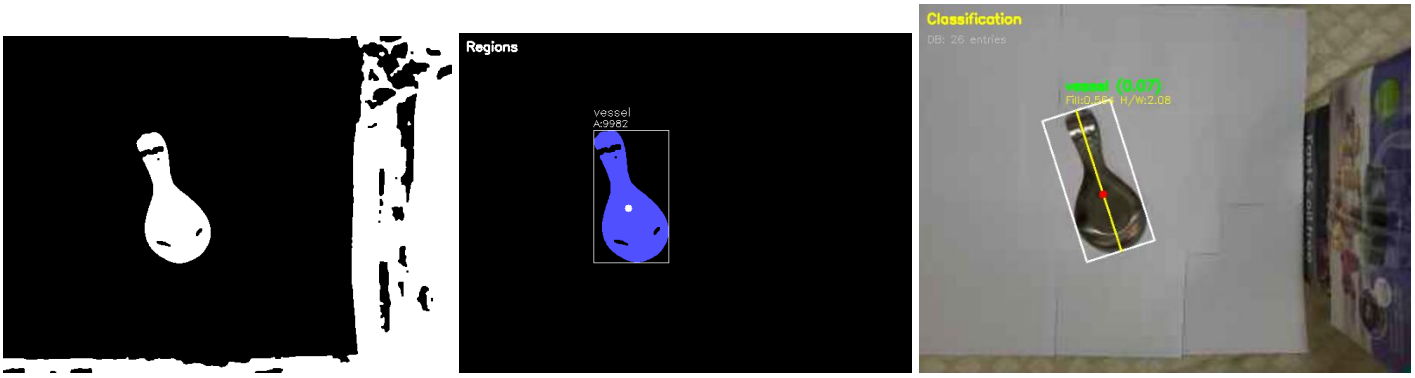
Task 5: Collecting training data

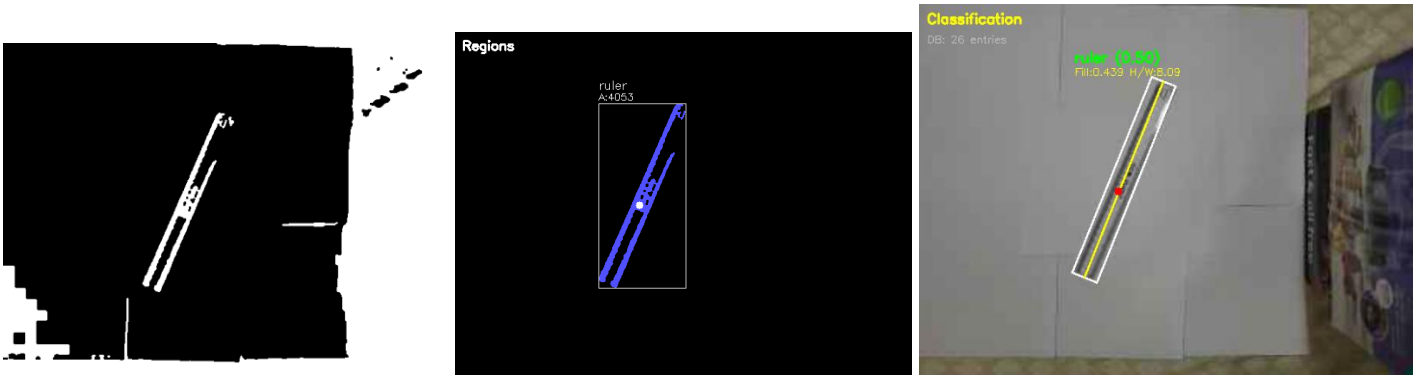
The training system works as follows: pressing ‘t’ during live video triggers a training prompt in the console. The feature vector of the largest detected region is extracted and written as a new row in training_data.csv along with the user-provided label. Multiple samples per object are collected at different positions and rotations to capture natural feature variation. The file persists between runs and is loaded automatically at startup for use during classification.

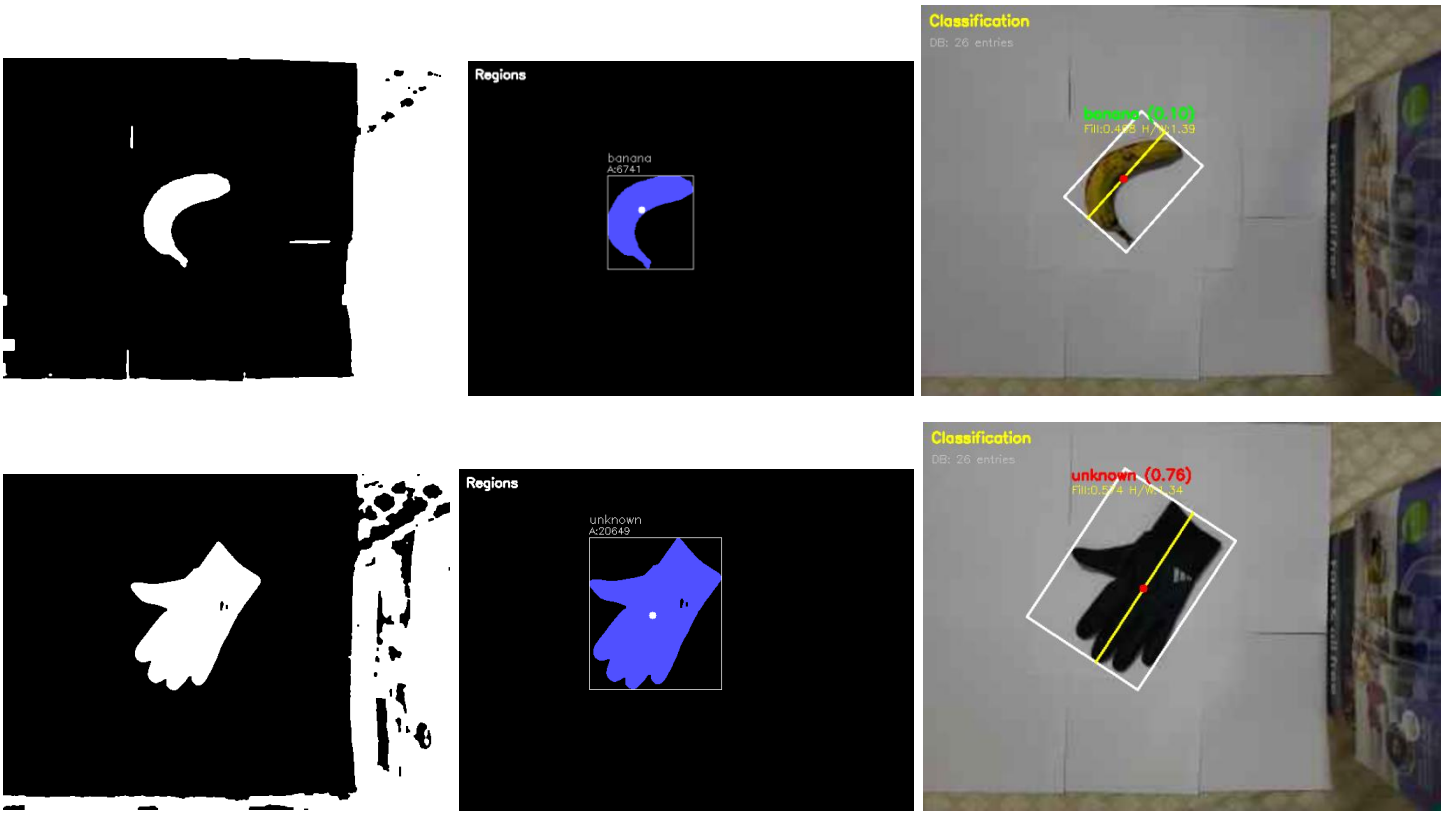
Task 6: Classifying new images











The figures show classification results for five objects across three columns: the cleaned binary image, the region map, and the classification output with the assigned label overlaid on the original frame.

Each correctly identified object displays its label in green above the oriented bounding box along with the nearest neighbor distance score in parentheses. The system correctly classifies the pouch, hard disk, vessel, ruler, and banana in their respective test images. The top right image shows the banana being correctly identified, while the bottom right image shows an untrained object being flagged as "unknown" in red, demonstrating the open-set rejection capability where objects not present in the training database are correctly rejected rather than misclassified.

Task 7: Evaluating the performance of the system

```
===== CONFUSION MATRIX =====
Rows = True Label | Columns = Predicted Label

             banana  harddisk     pouch     ruler    vessel
------------------------------------------------------------
    banana         3         0         0         0         0
  harddisk         0         3         0         0         0
     pouch         0         0         3         0         0
     ruler         0         0         0         3         0
    vessel         0         0         0         0         3

Total evaluated: 15
Correct: 15 / 15  Accuracy: 100.0%
============================
```

The confusion matrix shows results from 15 evaluation sessions across 5 object classes (3 per object). All 15 predictions were correct, giving 100% accuracy. Every diagonal entry equals 3 and all off-diagonal entries are 0, confirming that no object was confused with another. This strong performance is attributed to the objects having sufficiently distinct geometric shapes that the five hand-crafted features could separate them completely.

Task 8: Capture a demo of the system working

Link: **https://drive.google.com/file/d/1aYV7b2gyIGyhjtFZRw7sr0HbMhCgIUUf/view?usp=sharing**

Task 9: Implementing a one-shot classification system using image embeddings

How the Embedding Was Built,

Pre-processing:
Each detected region is pre-processed into a standardized input through four steps. First, the original frame is rotated by negative alpha around the region centroid, so the object's major axis aligns horizontally, removing rotational variation. Second, the axis-aligned bounding box extents (Dmin, Dmax, dmin, dmax) from the feature extraction step are used to crop the region as a plain rectangle. Third, the crop is resized to 224×224 pixels. Finally, pixel values are normalized using ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) values to match the preprocessing used during ResNet18's original training.
Network and Embedding Extraction

The pre-processed image is passed through a pre-trained ResNet18 ONNX model (resnet18-v2-7.onnx). Instead of using the final 1000-class classification output, the embedding is extracted from the global average pooling layer (onnx_node!resnetv22_pool1_fwd), which produces a 512-dimensional vector representing the visual content of the object. This layer was identified by inspecting all layer names using net.getLayerNames().

Training and Classification:
One training example per object is collected by running its pre-processed ROI through the network and saving the 512-dimensional embedding to embedding_db.csv. At classification time, the query embedding is compared against all stored embeddings using sum-squared difference (SSD). The label of the closest match is assigned to

the object. This is called one-shot classification because only a single example per class is required. Objects whose nearest neighbor SSD exceeds a set threshold are flagged as unknown.

Analysis,

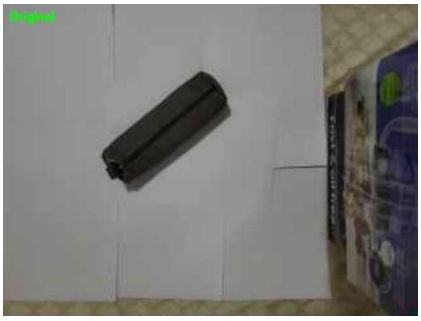
The hand-crafted feature system outperformed the ResNet18 embedding system on this particular object set, achieving 100% accuracy compared to 62.5% for the embedding approach. This result is explained by the nature of the objects chosen: hard disk, pouch, ruler, vessel, and watch have sufficiently distinct geometric shapes that five invariant features (H/W ratio, percent filled, eccentricity, and two Hu moments) were enough to separate them completely.

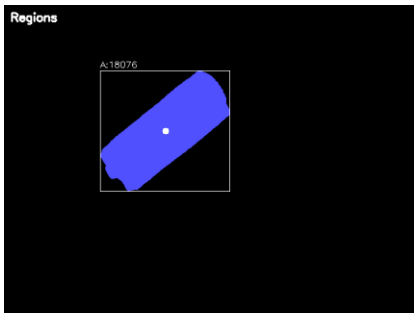
The embedding system struggled primarily with the hard disk and pouch classes, which were confused with each other in both directions. Both objects share a similar dark, flat, rectangular appearance, and after the ROI preprocessing step (rotation, cropping, and resizing to 224×224), the two objects look visually similar to the network. ResNet18 was pre-trained on ImageNet, which consists of natural photographs, and may not have developed strong discriminative features for distinguishing between two similarly shaped dark manufactured objects. Additionally, the one-shot nature of the embedding classifier (using only a single training example per class) leaves no tolerance for appearance variation due to lighting or viewing angle changes.

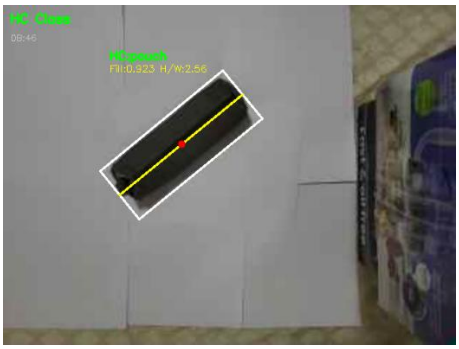
In general, the hand-crafted approach is better suited for datasets where objects differ primarily in geometric shape, while embedding-based approaches are expected to outperform hand-crafted features when objects are geometrically similar but visually distinct in texture or color, a condition not strongly present in this object set.

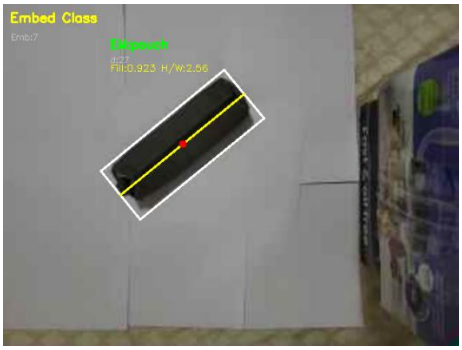
[Original Image] [Cleaned Threshold Image] [Threshold Region Image]

[Hand-Crafted Features Output] [Embedded Features Output]

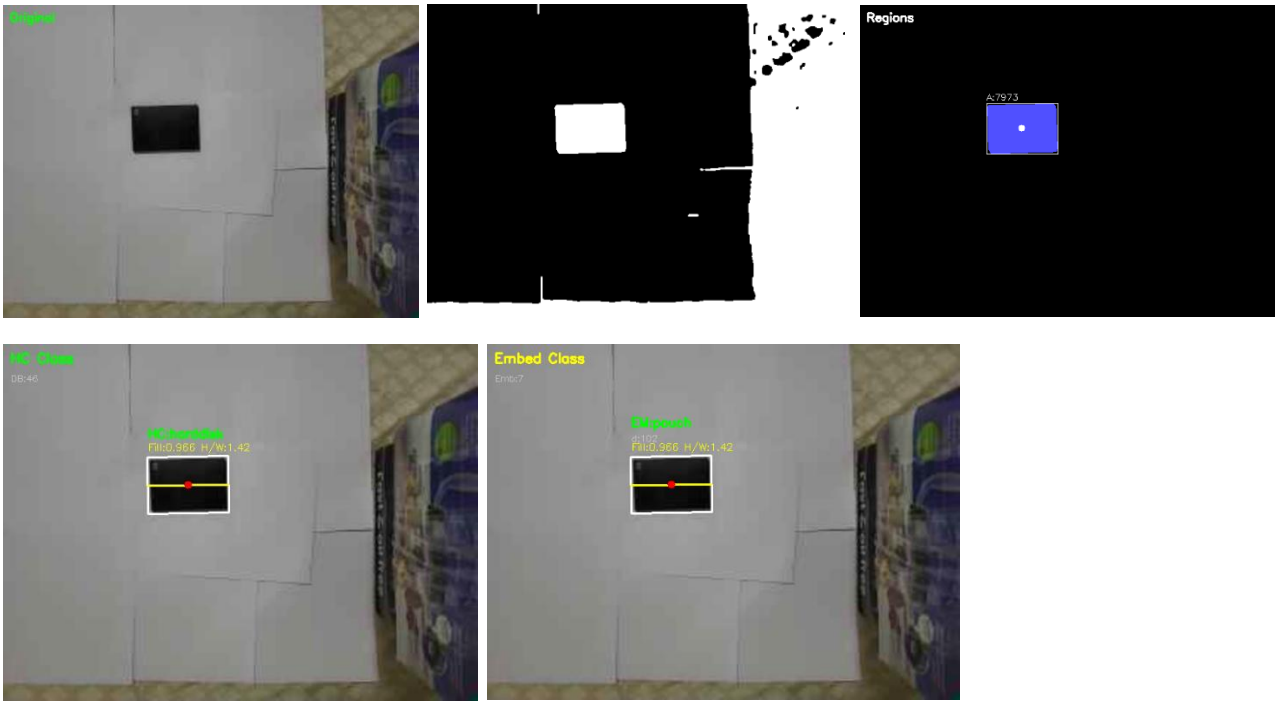


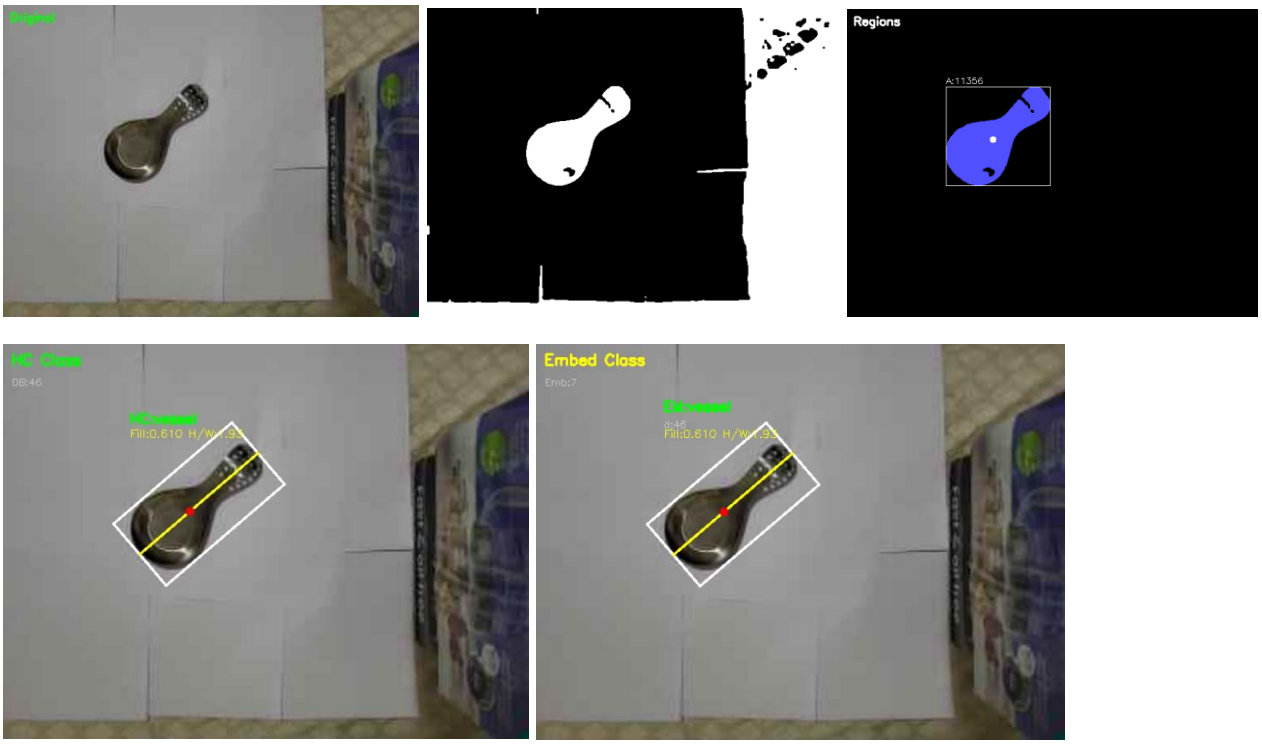




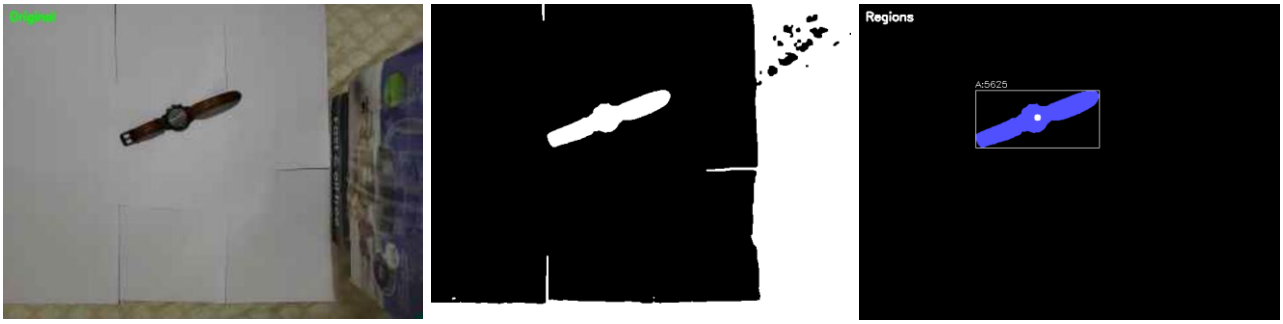


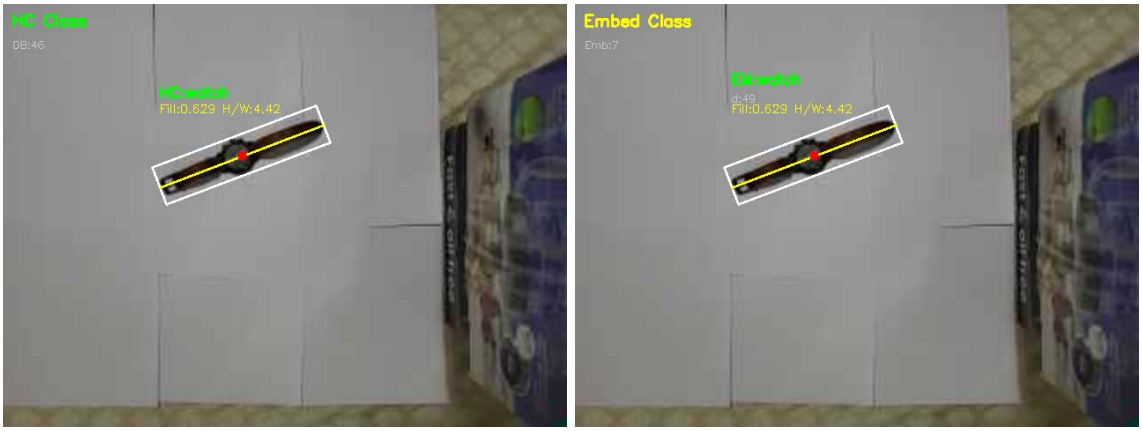
The top row shows the original frame, cleaned binary image, and region map with the pouch correctly segmented as a single blue region. Both the hand-crafted classifier and the embedding classifier correctly identify the object as "pouch", with their respective labels displayed in green above the oriented bounding box in the bottom two panels.

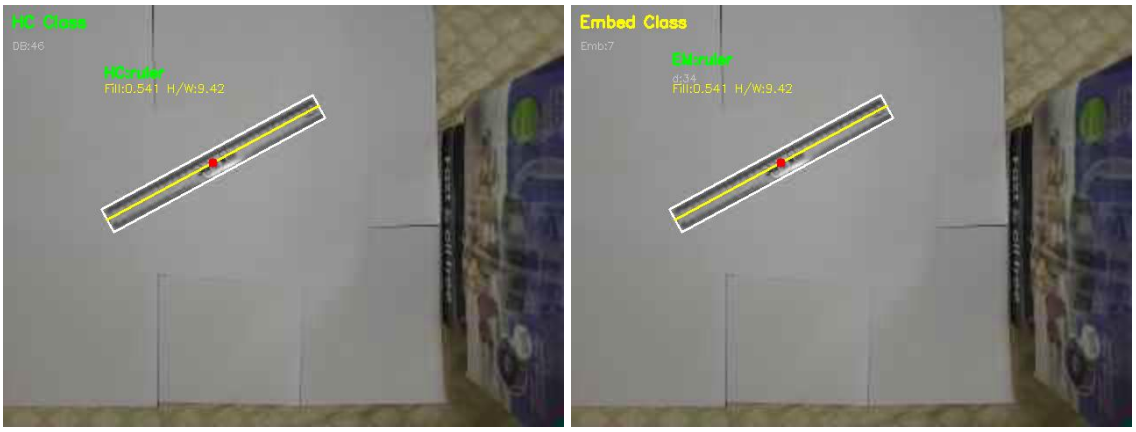




The first set shows the hard disk correctly classified as "hard disk" by both systems. The second set shows the vessel, where the hand-crafted system correctly labels it as "vessel" while the embedding system misclassifies it as a different object. This reflects the confusion seen in the embedding confusion matrix between visually similar dark objects.







The watch is correctly identified by both systems in the first set. The ruler, being a highly elongated thin object, is also correctly classified by both systems as shown in the second set. The oriented bounding box clearly captures the ruler's extreme elongation with a very high H/W ratio.

```
--- HAND-CRAFTED FEATURES ---

===== CONFUSION MATRIX =====
Rows=True | Cols=Predicted

             banana  harddisk     pouch     ruler    vessel     watch
------------------------------------------------------------------------
    banana         0         0         0         0         0         0
  harddisk         0         3         0         0         0         0
     pouch         0         0         4         0         0         0
     ruler         0         0         0         3         0         0
    vessel         0         0         0         0         3         0
     watch         0         0         0         0         0         3

Total: 16  Accuracy: 100.0%
============================

Confusion matrix saved to confusion_matrix_HC.csv
```

```
--- RESNET18 EMBEDDING ---

===== CONFUSION MATRIX =====
Rows=True | Cols=Predicted

             banana  harddisk     pouch     ruler    vessel     watch
------------------------------------------------------------------------
    banana         0         0         0         0         0         0
  harddisk         0         1         2         0         0         0
     pouch         0         2         2         0         0         0
     ruler         0         0         0         3         0         0
    vessel         0         0         0         0         2         1
     watch         0         1         0         0         0         2

Total: 16  Accuracy: 62.5%
============================

Confusion matrix saved to confusion_matrix_Emb.csv
```

The final image displays both confusion matrices printed onto the console. The hand-crafted feature system achieves 100% accuracy across 16 test samples with all predictions falling on the diagonal. The ResNet18 embedding system achieves 62.5% accuracy, with the primary confusion occurring between hard disk and pouch, and between vessel and watch. These confusions arise because visually similar dark objects produce nearby embeddings in the 512-dimensional feature space, particularly when only a single training example per class is available under the one-shot setup.

Reflection

This project provided hands-on experience building a complete computer vision pipeline from the ground up. Implementing thresholding and morphological filtering from scratch gave a deeper understanding of how binary image analysis works at the pixel level, beyond simply calling library functions. Working with connected components and region-based feature extraction made the theoretical concepts of moments, central axes, and oriented bounding boxes much more concrete and intuitive.

One of the most valuable lessons was understanding why feature invariance matters, watching feature values remain stable while moving and rotating an object in real time made the concept click in a way that textbook descriptions alone could not. The comparison between hand-crafted features and deep learning embeddings was particularly insightful, as it demonstrated that more complex models do not always outperform simpler ones, and that the choice of method should be driven by the specific characteristics of the problem.

Setting up a real-time pipeline also highlighted practical engineering challenges such as camera latency, lighting sensitivity, and threshold tuning that are rarely discussed in theory but are critical in practice.

Acknowledgements

The following materials and resources were consulted during this assignment:

- Course textbook: Maxwell, B.A. *Fundamentals of Computer Vision* (2022 draft), for concepts on thresholding, morphological processing, connected components, and 2D shape descriptors including moments and oriented bounding boxes.
- OpenCV documentation (docs.opencv.org): For usage of connectedComponentsWithStats, cv::moments, cv::dnn::readNetFromONNX, and related functions.
- ONNX Model Zoo (github.com/onnx/models): For the pre-trained resnet18-v2-7.onnx model used in the embedding classification task.
- Professor-provided utilities code and ResNet18 network file: Used as the basis for the embedding preprocessing pipeline and network loading, with attribution as required.
- Claude (Anthropic): Used as an AI assistant to help structure, debug, and implement the C++ codebase throughout the project.